DAY 6 · CALLBACKS & EVENT LOOP

THEORY & NOTES

Geeta University · Summer Internship 2026

2	6	5	Async JS Foundations
Week	Day	Topics	Focus

Topic 1 — Synchronous vs Asynchronous Execution

JavaScript ek **single-threaded** language hai — ek samay mein sirf ek kaam kar sakta hai. **Synchronous** code line-by-line execute hota hai — ek line complete hone ke baad hi agla chalta hai. **Asynchronous** code time-consuming tasks ko background mein handle karta hai aur baki code ko rokta nahi — callback ya promise ke zariye result baad mein milta hai.

Synchronous	Asynchronous	Single Thread	Non-blocking I/O
Line-by-line, blocking. Ek time pe ek kaam. Simple lekin slow tasks block karta hai.	Non-blocking. Time-consuming tasks background mein. Main thread free rehta hai.	JS ka ek hi execution thread hai — parallel nahi, concurrent hai (event loop ki wajah se).	Network, file, timer — sab browser/Node ke Web APIs handle karte hain, JS nahi.

Synchronous (Blocking)

```
// SYNCHRONOUS — Blocking
console.log('1: Start');

function heavyTask() {
// Imagine 3 second computation
let i = 0;
while(i < 1e9) i++; // blocks!
return 'done';
}

console.log('2:', heavyTask());
console.log('3: End');
// Output: 1 -> waits 3s -> 2 -> 3
// UI freezes during heavyTask!
// User can't click/scroll/type
```

Asynchronous (Non-blocking)

```
// ASYNCHRONOUS — Non-blocking
console.log('1: Start');

// setTimeout is async — doesn't block
setTimeout(() => {
console.log('3: Timer done');
}, 2000);

// fetch is async — network in background
fetch('/api/data')
.then(r => r.json())
.then(data => console.log('4:', data));

console.log('2: End of sync code');
// Output: 1 -> 2 -> (2s later) -> 3,4
// UI stays responsive throughout!
```

Topic 2 — Call Stack, Web APIs & Callback Queue

JavaScript runtime mein teen main parts hain: **Call Stack** — jahan code execute hota hai (LIFO — Last In First Out), **Web APIs** — browser provided async capabilities (setTimeout, fetch, DOM events), **Callback Queue (Task Queue)** — completed async tasks ke callbacks wait karte hain Call Stack khaali hone ka.

Call Stack
LIFO structure. Sync code execute hota hai. Jab function call ho — push. Return — pop.

Web APIs
Browser ka part (JS nahi) — setTimeout, setInterval, fetch, addEventListener sab Web APIs hain.

Callback Queue
Macrotask queue. setTimeout/setInterval callbacks yahan wait karte hain.

Microtask Queue
Higher priority — Promise .then(), queueMicrotask(). ALWAYS callback queue se pehle drain hoti hai.

Call Stack — LIFO Execution

```
// Step-by-step Call Stack trace
function multiply(a, b) { return a * b; } // Step 3: push multiply
function square(n) { return multiply(n, n); } // Step 2: push square
function printSquare(n) {
const result = square(n); // Step 1: push printSquare -> calls square
console.log(result);
}
printSquare(5);
// Call Stack sequence:
// [printSquare] -> [printSquare, square] -> [printSquare, square, multiply]
// -> multiply returns 25 -> [printSquare, square] -> square returns 25
// -> [printSquare] -> console.log(25) -> printSquare returns -> []
// Stack Overflow — infinite recursion:
function boom() { boom(); } // Each call pushes to stack
// boom(); // Eventually: RangeError: Maximum call stack size exceeded
```

Web APIs + Queue Priority — Critical!

```
// Web APIs hand-off + Callback Queue
console.log('1: Script start');
// setTimeout hands off to Web API (browser's timer mechanism)
setTimeout(() => {
console.log('4: Timeout callback');
}, 0); // 0ms delay but STILL async!
// fetch hands off to Web API (browser's network mechanism)
fetch('https://jsonplaceholder.typicode.com/todos/1')
.then(r => r.json())
.then(data => console.log('5: Fetch data:', data.title));
console.log('2: Script end (sync)');
Promise.resolve().then(() => console.log('3: Microtask!'));
// OUTPUT ORDER: 1 -> 2 -> 3 -> 4 -> 5
// Why 3 before 4? Microtask queue (Promise) drains BEFORE macrotask queue (setTimeout)!
// RULE: Sync Code > Microtasks (Promises) > Macrotasks (setTimeout, setInterval)
```

■ Critical: setTimeout(fn, 0) is NOT Immediate

setTimeout(fn, 0) — 0ms delay matlab INSTANT nahi hai!

Callback Callback Queue mein jaata hai — sirf tab chalega jab Call Stack BILKUL khaali ho aur Microtasks drain ho jayein.

Yahi reason hai ki: console.log('A'); setTimeout(()=>console.log('B'),0); console.log('C');

Output: A -> C -> B (never A -> B -> C!)

Production bug: async se milne wale result ko sync code mein use karne ki koshish — undefined milega.

Topic 3 — Event Loop Mechanics (Visual Walkthrough)

Event Loop ek continuous loop hai jo check karta rehta hai: kya Call Stack khaali hai? Agar haan, to Microtask Queue se koi task hai? Agar haan to uthao. Microtasks khatam — Callback Queue se ek task uthao. Ye loop browser mein hamesha chalta rehta hai.

Event Loop — Step by Step Trace

```
// ■■ COMPLETE EVENT LOOP WALKTHROUGH ■■■■■■■■■■■■■■■■■■■■■■■■■■
// Trace exactly what happens, step by step
console.log('START'); // 1. Sync — runs immediately
setTimeout(() => console.log('TIMEOUT 1'), 0); // 2. Web API: timer starts (0ms)
setTimeout(() => console.log('TIMEOUT 2'), 0); // 3. Web API: timer starts (0ms)
Promise.resolve()
.then(() => {
console.log('PROMISE 1'); // Microtask queue
return Promise.resolve();
})
.then(() => console.log('PROMISE 2')); // Chained microtask
queueMicrotask(() => console.log('MICROTASK'));// Directly to microtask queue
console.log('END'); // 4. Sync — runs immediately
// ■■ EXACT OUTPUT ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// START
// END
// PROMISE 1 <- Microtask queue (before macrotasks!)
// MICROTASK <- Microtask queue
// PROMISE 2 <- Chained promise (new microtask created during PROMISE 1)
// TIMEOUT 1 <- Macrotask (callback queue)
// TIMEOUT 2 <- Macrotask (callback queue)
// EVENT LOOP ALGORITHM:
// 1. Run all sync code (fill call stack and empty it)
// 2. Drain ENTIRE microtask queue (including newly added microtasks!)
// 3. Pick ONE macrotask from callback queue
// 4. Go to step 2 (check microtasks again after each macrotask)
```

Microtask Starvation — Edge Case

```
// Proving microtasks drain completely before each macrotask
setTimeout(() => console.log('setTimeout'), 0);
Promise.resolve()
.then(() => {
console.log('Promise 1');
// Adding ANOTHER microtask from inside a microtask
Promise.resolve().then(() => console.log('Promise 1.1 (nested)'));
})
.then(() => console.log('Promise 2'));
```

```
// Output:
// Promise 1
// Promise 1.1 (nested) <- nested microtask runs before setTimeout!
// Promise 2
// setTimeout <- macrotask runs LAST
// KEY INSIGHT: If you keep adding microtasks inside microtasks,
// setTimeout NEVER runs — starvation of macrotask queue!
```

■ Real World Impact — Why Event Loop Matters

React setState batching — multiple state updates ek event loop iteration mein batch hote hain.

Node.js — same event loop but different Web APIs (libuv): fs, net, child_process.

Long sync tasks — 1 second loop JS mein = 1 second UI freeze (no events processed!).

requestAnimationFrame — next paint se pehle chalta hai — smooth animations ke liye.

Interview mein: 'JavaScript async kaise handle karta hai single thread pe?' — Event Loop!

Topic 4 — Callback Pattern & Callback Hell

Callback ek function hai jo doosre function ko argument ke roop mein pass kiya jaata hai aur baad mein (asynchronously) call hota hai. Ye sabse purana async pattern hai. **Callback Hell** (aka Pyramid of Doom) tab hota hai jab nested callbacks ka level bahut deep ho jaata hai — code padhna aur debug karna mushkil ho jaata hai.

Callback Pattern — Sync & Async

```
// ■■ BASIC CALLBACK PATTERN ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Synchronous callback (forEach, map, sort use these)
const nums = [3, 1, 4, 1, 5];
nums.forEach(n => console.log(n)); // callback called sync
nums.sort((a, b) => a - b); // comparator = callback
// Asynchronous callback
function fetchUser(id, onSuccess, onError) {
setTimeout(() => {
if (id > 0) {
onSuccess({ id, name: `User ${id}` }); // success callback
} else {
onError(new Error('Invalid ID')); // error callback
}
}, 1000);
}
fetchUser(1,
user => console.log('Got user:', user.name), // onSuccess
error => console.log('Error:', error.message) // onError
);
console.log('This runs BEFORE the callback!');
// Node.js Error-First Callback Convention
// First arg = error (null if success), second = result
fs.readFile('./data.txt', 'utf8', (err, data) => {
```

```
if (err) { console.error(err); return; }
console.log(data);
});
```

Callback Hell (avoid)

```
// ■ CALLBACK HELL
// Read file -> parse -> fetch user
// -> save to DB (nested 4 levels!)
readFile('users.txt', (err, data) => {
if (err) { handle(err); return; }
parseData(data, (err, users) => {
if (err) { handle(err); return; }
fetchDetails(users[0], (err, info) => {
if (err) { handle(err); return; }
saveToDb(info, (err, result) => {
if (err) { handle(err); return; }
console.log('Done:', result);
// Keep nesting....
});
});
});
});
// PROBLEMS:
// - Hard to read (pyramid shape)
// - Error handling repeated
// - Hard to test each step
```

Promise / Async-Await (better)

```
// ■ SAME FLOW with Promises
// (Day 7 topic — teaser!)
readFile('users.txt')
.then(data => parseData(data))
.then(users => fetchDetails(users[0]))
.then(info => saveToDb(info))
.then(result => {
console.log('Done:', result);
})
.catch(err => handle(err));
// ONE catch handles all errors!
// ■ EVEN BETTER with async/await
// (Day 8 topic)
try {
const data = await readFile('users.txt');
const users = await parseData(data);
const info = await fetchDetails(users[0]);
const result= await saveToDb(info);
console.log('Done:', result);
} catch (err) { handle(err); }
```

Escape Callback Hell — Named Functions (without Promises)

```
// Fix Callback Hell WITHOUT Promises — Named Functions technique
// Break each step into named functions — flat, readable
function handleSaveResult(err, result) {
if (err) return handleError(err);
console.log('Done:', result);
}
function handleFetchedInfo(err, info) {
if (err) return handleError(err);
saveToDb(info, handleSaveResult);
}
function handleParsedUsers(err, users) {
if (err) return handleError(err);
fetchDetails(users[0], handleFetchedInfo);
}
function handleFileData(err, data) {
if (err) return handleError(err);
parseData(data, handleParsedUsers);
}
function handleError(err) { console.error('Error:', err.message); }
```

```
readFile('users.txt', handleFileData); // clean, flat, readable!
// Downside: lots of named functions, harder to follow flow upward
```

Topic 5 — setTimeout / setInterval Internals

setTimeout(fn, delay) — fn ko minimum delay milliseconds ke baad ek baar call karta hai. **setInterval(fn, delay)** — fn ko har delay milliseconds ke baad bar bar call karta hai. Dono Web APIs hain — timer browser/Node.js handle karta hai, JS nahi. Return value ek **timer ID** hai jo clearTimeout/clearInterval se cancel kar sakte hain.

setTimeout & setInterval — Basics + Common Mistakes

```
// ■■ setTimeout ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Basic
const timerId = setTimeout(() => console.log('After 2s'), 2000);
// Cancel before it fires
clearTimeout(timerId); // timer cancelled — callback never runs
// Passing arguments to callback
setTimeout((name, role) => console.log(`${name} is ${role}`), 1000, 'Rahul', 'Admin');
// setTimeout returning value — COMMON MISTAKE
function delayedValue() {
let result;
setTimeout(() => { result = 42; }, 1000);
return result; // ■ returns undefined! setTimeout hasn't fired yet
}
console.log(delayedValue()); // undefined — NOT 42
// ■■ setInterval ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
let count = 0;
const intervalId = setInterval(() => {
count++;
console.log(`Tick ${count}`);
if (count === 5) {
clearInterval(intervalId); // stop after 5 ticks
console.log('Stopped!');
}
}, 1000);
// setInterval drift problem — callback takes time too
// If callback takes 500ms and interval is 1000ms,
// actual interval = 1000ms + 500ms = 1500ms (drifts!)
```

Advanced Patterns — Recursive setTimeout, Debounce, Throttle

```
// ■■ SELF-CORRECTING INTERVAL using recursive setTimeout ■■■■■■■■
// Better than setInterval for long-running tasks
function accurateInterval(fn, delay) {
let start = Date.now();
let expected = start + delay;
let timerId;
function tick() {
```

```
const drift = Date.now() - expected;
fn(); // call the actual function
expected += delay;
// Adjust next timeout to compensate for drift
timerId = setTimeout(tick, Math.max(0, delay - drift));
}
timerId = setTimeout(tick, delay);
return () => clearTimeout(timerId); // return cancel function
}
const stop = accurateInterval(() => console.log('accurate tick'), 1000);
setTimeout(stop, 5500); // stop after ~5 ticks
// ■■ DEBOUNCE using setTimeout (Day 5 concept revisit) ■■■■■■■■■■
function debounce(fn, delay) {
let timerId;
return function(...args) {
clearTimeout(timerId); // cancel previous timer every call
timerId = setTimeout(() => fn.apply(this, args), delay);
};
}
// ■■ THROTTLE using setTimeout ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
function throttle(fn, limit) {
let inThrottle = false;
return function(...args) {
if (!inThrottle) {
fn.apply(this, args);
inThrottle = true;
setTimeout(() => { inThrottle = false; }, limit);
}
};
}
const debouncedSearch = debounce(search, 400); // fires 400ms after last call
const throttledScroll = throttle(onScroll, 100); // fires at most once per 100ms
```

■ Critical: setInterval vs Recursive setTimeout

setInterval — callback ke chalte waqt bhi timer chalta rehta hai (drift + overlap possible).

Agar callback time > interval, agle interval ke time hi nayi call stack pe push — pileup!

Recursive setTimeout — har baar naya timer sirf callback complete HONE KE BAAD start hota hai.

APIs poll karne ke liye: recursive setTimeout prefer karo — no overlap, no drift.

Memory leak: setInterval ka reference clear nahi kiya to component unmount ke baad bhi chalta rehta hai (React mein common bug!).

Day 6 Learning Outcomes

Sync vs async ka fundamental difference explain kar paoge.

Call Stack, Web APIs, Callback Queue, aur Microtask Queue ka flow draw kar paoge.

Event loop ka exact algorithm bata paoge — microtask vs macrotask priority.

Callback pattern likhna aayega — error-first Node.js convention ke saath.

Callback Hell identify aur named functions se fix kar paoge.

setTimeout/setInterval ke timing traps avoid kar paoge, debounce/throttle implement kar paoge.